Formal Methods for Security and Privacy (SS26) Prof. Matteo Maffei

Project 4 – Machine Learning Verification

Due | *15.07.2026* TU Wien

1 General Remarks

This project focuses on verifying safety properties in fully connected feed forward neural networks. Specifically, you are asked to encode and verify robustness and reachability properties using Marabou.

2 Setting up Your System

This homework requires verifying your answers using Marabou Neural Network Verifier. You can install Marabou by following the steps below or refer to the Marabou repository for more information.

Marabou build dependencies. The build process uses CMake version 3.16 (or later). You can get CMake here or you can install it using the following commands (debian/ubuntu):

```
# this installs the essential build tools (gcc and related components)
sudo apt install build-essential
sudo apt install cmake
```

Next, clone the repository and build the project as follows:

```
git clone https://github.com/NeuralNetworkVerification/Marabou/
cd Marabou/
mkdir build
cd build
cmake ../
cmake --build ./
```

Troubleshooting. If you encounter an Openblas dependency error, do the following:

```
cd /Marabou/build/tools/OpenBLAS-0.3.19/
sudo apt install gfortran
make TARGET=HASWELL -j$(nproc)
make PREFIX=/root/FMSP/Marabou/tools/OpenBLAS-0.3.19/installed install
```

And now run `cmake ..` under `Marabou/build` again

3 Submission Instructions

Please submit your solution by *15.07.2026* on TUWEL. This project will account for 18 out of the 60 project points.

Your submission must be a single archive file `name-matriculation.zip` containing

1. The `submission.pdf` file containing all your answers and explanations, a LaTeX template can be found on TUWEL.
2. The text files containing your solutions, as described in the submission box of each task

Required files
`ACASXU_property1.txt`
`ACASXU_property2.txt`

4 Collision Avoidance

Airborne collision avoidance systems are vital for aviation safety. The original Traffic Alert and Collision Avoidance System, developed after midair collisions, is now mandatory on all large commercial aircraft globally. More recently, Airborne Collision Avoidance System (ACAS Xu) has emerged, using advanced decision-making to minimize collisions and unnecessary alerts.

The unmanned version, ACAS Xu, provides horizontal maneuver advice. Traditionally, it relied on a 2GB lookup table of sensor measurements, which was too large for certified aircraft hardware. To solve this, Deep Neural Networks (DNNs) were introduced as a replacement. This not only drastically cut memory requirements to under 3MB but also improved safety and performance due to the DNNs continuous nature. To further speed up lookup times, the system now uses an array of 45 individual DNNs.

However, using DNNs in ACAS Xu creates new certification hurdles. Proving that these continuous DNNs would not produce dangerous or incorrect alerts is absolutely crucial for safety-critical applications. Past methods, like 1.5 million simulated tests, are no longer sufficient to guarantee the absence of errors. This highlights the need for verifying DNNs and makes the ACAS Xu DNNs prime candidates on which to apply Marabou.

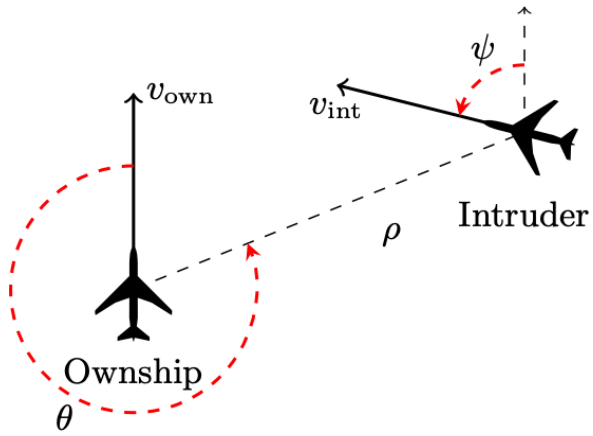

Figure 1: Geometry for ACAS Xu Logic.

The ACAS Xu system maps input variables to action advisories. Each advisory is assigned a score, with the lowest score corresponding to the best action. The input state is composed of five dimensions (shown in Fig. 1) which represent information determined from sensor measurements [1]:

- ρ: Distance from ownship to intruder;
- θ: Angle to intruder relative to ownship heading direction;
- ψ: Heading angle of intruder relative to ownship heading direction;
- v_{own}: Speed of ownship;
- v_{int}: Speed of intruder.

There are five outputs which represent the different horizontal advisories that can be given to the ownship:

- Clear-of-Conflict (COC),
- weak right,
- strong right,
- weak left,
- strong left.

Weak and strong mean heading rates of 1.5°/s and 3.0°/s, respectively.

The inputs are denoted by variables `x0`, `x1`, …`x4` and the outputs by `y0, y1`, …`y4` respectively in the .nnet model. We want to verify the two reachability properties described in the next section.

Hint: This is a sample property file (`.txt`) with a single input and single output in Marabou:

```
x0 >= 0.5
x0 <= 0.7
y0 >= 0
```

4.1 Project Task

Encode the following Properties 1 and 2 in their respective .txt files and verify them on the specific networks indicated below using Marabou.

1. (1 point) **Property 1 (Large Vertical Separation / No Strong Right)**: The ACAS Xu system uses different neural networks depending on the current vertical separation and previous advisories. Network `1_9` is used when vertical separation is large. In this state, the network should *never* advise a strong turn.

To prevent execution timeouts, we will test a highly specific subset of this property. If vertical separation is large, the network will never advise a strong right turn, i.e. the score for a **Strong Right** turn will never be the minimal score.

Encode this property in `ACASXU_property1.txt` and verify it on the network `resources/nnet/acasxu/ACASXU_experimental_v2a_1_9.nnet`.

Normalized Input Constraints:

$$\begin{aligned} 0.50000 \leq x0 \leq 0.60000 \\ -0.05000 \leq x1 \leq 0.05000 \\ -0.05000 \leq x2 \leq 0.05000 \\ -0.01000 \leq x3 \leq 0.01000 \\ -0.01000 \leq x4 \leq 0.01000 \end{aligned}$$

2. (1 point) **Property 2 (The Distant Intruder)**: If the intruder is extremely distant and is significantly slower than the ownship, the score of a COC advisory will **never** be the maximal score (i.e., it should never be considered the worst possible action). Encode this property in `ACASXU_property2.txt` and verify it on a different network slice: `resources/nnet/acasxu/ACASXU_experimental_v2a_2_1.nnet`.

$$0.60000 \leq x0 \leq 0.67986$$
$$-0.50000 \leq x1 \leq 0.50000$$
$$-0.50000 \leq x2 \leq 0.50000$$
$$0.45000 \leq x3 \leq 0.50000$$
$$-0.50000 \leq x4 \leq -0.45000$$

3. (4 points) Based on the verification results you obtained, answer the following:

- What was the outcome of Property 1 on network `1_9`? What does this tell us about the network's behavior in a large vertical separation scenario?
- What was the outcome of Property 2 on network `2_1`? Explain what this result physically means for the aircraft. Why is discovering this specific outcome important in the context of certifying Deep Neural Networks for aviation?

Submission (6pt). Add to your submission the `ACASXU_property1.txt` and `ACASXU_property2.txt` files containing the encoding of the above properties. Discuss in Sec. 4.1 of your `submission.pdf` report the encoding of the properties and your answer to question 3.

5 MNIST Robustness: Network Size and Scalability

The MNIST dataset is a widely used collection of handwritten digits (0-9) used for training and testing image classification algorithms. It consists of 60,000 training images and 10,000 test images, each being a 28×28 pixel grayscale image.

In the Marabou repository, you will find different, trained MNIST networks of varying sizes. You will also find property files corresponding to different images and different ϵ (epsilon) perturbation bounds.

These property files encode a robustness verification problem. The input variables represent the 784 pixel values of the image, constrained within an ϵ-ball around the original image. The goal is to see if adding this slight layer of "noise" can trick the network into misclassifying the digit into a specific wrong target.

5.1 Project Task

For this task, you will explore how both the size of the adversarial perturbation (ϵ) and the size of the neural network architecture affect the safety and verifiability of the system. Set the timeout to 3600 sec.

1. (3 points) **The Epsilon Spectrum:** Locate the property files for Image 2 targeted at label 1 in the repository. Run Marabou on the **smaller** network (`mnist10x10.nnet`) for all three available ϵ values:
   - `image2_target1_epsilon0.005.txt`
   - `image2_target1_epsilon0.05.txt`
   - `image2_target1_epsilon0.1.txt`

Report the outcome (SAT, UNSAT or timeout) for each ϵ. Explain what this progression tells you about the network's robustness boundary.

2. (3 points) **The Architecture Comparison:** Now, take the exact same three property files from 5.1.1 and verify them against the **larger** MNIST network (`mnist6x256.nnet`) provided in the repository.
    - Report the outcomes (SAT, UNSAT or timeout) for the larger network. Did its robustness boundary differ from the smaller network?
    - Record and compare the execution time for both networks at $\epsilon = 0.005$, 0.05 and 0.1.
3. (4 points) **Interpretation:** Based on your experiments, explain the relationship between network size, execution time, and robustness. If you were deploying a safety-critical image classifier on an aircraft, how would you balance the need for a highly accurate (large) network with the need for a verifiable (small) network?

Submission (10pt). Discuss in Sec. 5.1 of the `submission.pdf` report your answers to the above questions. Be sure to clearly state your SAT/UNSAT results and your execution time comparisons.

6 Final Synthesis

6.1 Project Task

1. (2 points) **Reachability vs. Robustness**: In this project, you verified **reachability (safety) properties** for the ACAS Xu network and **robustness properties** for the MNIST network.

Based on the `.txt` files you used and created, explain the fundamental difference between these two types of verification. Specifically, how does the definition of the input and output space (the `x` constraints) differ when verifying a general safety rule (like ACAS Xu) versus verifying adversarial robustness (like MNIST)?

Submission (2pt). Discuss in the final section of your `submission.pdf` report your answer to the synthesis question.

References

[1] G. Katz, C. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer, "Reluplex: An efficient SMT solver for verifying deep neural networks," in *International conference on computer aided verification*, Springer, 2017, pp. 97–117.